

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Mohamed Jameel A	Reg. No.:	192425057
Section / Batch:		Date:	

Code of Conduct

I Mohamed Jameel A / 192425057 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select $N = 1, 2$, or 3 , display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.

Code:

```
import re
```

```
from collections import Counter, defaultdict
```

```
# -----
```

```
# Q1: N-GRAM BASED NEXT-WORD PREDICTION
```

```
# -----
```

```
corpus = ""
```

```
Artificial intelligence is transforming the world.
```

```
Artificial intelligence is changing modern technology.
```

```
Artificial intelligence is useful in many applications.
```

```
Artificial intelligence is becoming more powerful.
```

```
Machine learning is a branch of artificial intelligence.
```

```
Machine learning can analyze large amounts of data.
```

```
Machine learning can solve complex problems.
```

```
Deep learning is a part of machine learning.
```

```
Deep learning is used in computer vision.
```

```
Deep learning is used in natural language processing.
```

```
Natural language processing is a field of artificial intelligence.
```

```
Natural language processing helps computers understand language.
```

```
Natural language processing is useful for text analysis.
```

```
Language models predict the next word in a sentence.
```

```
Language models are important in natural language processing.
```

```
A language model learns patterns from a text corpus.
```

```
The model predicts words using probability.
```

```
The model can generate natural language.
```

```
Smart text input can predict the next word.
```

```
Smart text input helps users type faster.
```

```
Artificial intelligence and machine learning are important technologies.
```

```
""
```

```
# -----
```

```
# 1. PREPROCESSING
```

```
# -----
```

```
def tokenize(text):
```

```
    text = text.lower()
```

```
    return re.findall(r"\b[a-z]+\b", text)
```

```
sentences = re.split(r"[.!?]+", corpus)
```

```
tokenized_sentences = []
```

```
for sentence in sentences:
```

```
    words = tokenize(sentence)
```

```
    if words:
```

```
        tokenized_sentences.append(["<s>"] + words + ["</s>"])
```

```
# -----
```

```
# 2. BUILD N-GRAM COUNTS
```

```
# -----
```

```
unigram_counts = Counter()
```

```
bigram_counts = Counter()
```

```
trigram_counts = Counter()
```

```
for sentence in tokenized_sentences:
```

```
    # Unigrams
```

```

for word in sentence:
    unigram_counts[word] += 1
# Bigrams
for i in range(len(sentence) - 1):
    bigram = (sentence[i], sentence[i + 1])
    bigram_counts[bigram] += 1
# Trigrams
for i in range(len(sentence) - 2):
    trigram = (sentence[i], sentence[i + 1], sentence[i + 2])
    trigram_counts[trigram] += 1
# -----
# 3. PROBABILITY FUNCTIONS
# -----

def unigram_probability(word):
    total = sum(unigram_counts.values())
    return unigram_counts[word] / total if total else 0
def bigram_probability(previous_word, word):
    denominator = unigram_counts[previous_word]
    if denominator == 0:
        return 0
    return bigram_counts[(previous_word, word)] / denominator
def trigram_probability(word1, word2, word3):
    denominator = bigram_counts[(word1, word2)]
    if denominator == 0:
        return 0
    return trigram_counts[(word1, word2, word3)] / denominator
# -----
# 4. NEXT-WORD PREDICTION
# -----

def predict_next_words(sentence, n=3, top_k=5):
    words = tokenize(sentence)
    candidates = []
    if n == 1:
        for word, count in unigram_counts.items():
            if word in ["<s>", "</s>"]:
                continue
            probability = unigram_probability(word)
            candidates.append((word, probability))
    elif n == 2:
        if len(words) == 0:
            return []
        previous_word = words[-1]
        for (w1, w2), count in bigram_counts.items():
            if w1 == previous_word and w2 not in ["<s>", "</s>"]:
                probability = bigram_probability(w1, w2)
                candidates.append((w2, probability))
    elif n == 3:
        if len(words) < 2:
            return []
        w1 = words[-2]
        w2 = words[-1]

        for (x1, x2, x3), count in trigram_counts.items():
            if x1 == w1 and x2 == w2 and x3 not in ["<s>", "</s>"]:

```

```

        probability = trigram_probability(x1, x2, x3)
        candidates.append((x3, probability))
    candidates.sort(key=lambda x: x[1], reverse=True)
    return candidates[:top_k]
# -----
# 5. DISPLAY N-GRAM COUNTS
# -----
print("=" * 60)
print("N-GRAM LANGUAGE MODEL")
print("=" * 60)
print("\nTop Unigrams:")
for gram, count in unigram_counts.most_common(10):
    print(gram, ":", count)
print("\nTop Bigrams:")
for gram, count in bigram_counts.most_common(10):
    print(gram, ":", count)
print("\nTop Trigrams:")
for gram, count in trigram_counts.most_common(10):
    print(gram, ":", count)
# -----
# 6. PREDICTION FOR GIVEN SENTENCE
# -----
test_sentence = "Artificial intelligence is"
print("\n" + "=" * 60)
print("PREDICTION")
print("=" * 60)
print("\nInput sentence:", test_sentence)
for n in [1, 2, 3]:
    predictions = predict_next_words(test_sentence, n)
    print(f"\nN = {n}")
    if predictions:
        for word, probability in predictions:
            print(f"{word:15s} Probability = {probability:.4f}")
    else:
        print("No prediction available.")
# -----
# 7. UNSEEN N-GRAM DEMONSTRATION
# -----
print("\n" + "=" * 60)
print("UNSEEN N-GRAM TEST")
print("=" * 60)
unknown = ("quantum", "banana", "computer")
print(
    "Probability of unseen trigram",
    unknown,
    "=",
    trigram_probability(*unknown)
)
# -----
# 8. SIMPLE PREDICTION ACCURACY
# -----
test_cases = [
    ("Artificial intelligence is", "transforming"),
    ("Machine learning is", "a"),
    ("Deep learning is", "a"),

```

```

    ("Natural language processing is", "a"),
    ("Language models predict", "the")
]
correct = 0
total = len(test_cases)
print("\n" + "=" * 60)
print("PREDICTION ACCURACY")
print("=" * 60)
for sentence, expected in test_cases:
    predictions = predict_next_words(sentence, n=3)
    predicted_words = [word for word, prob in predictions]
    print("\nInput:", sentence)
    print("Expected:", expected)
    print("Predicted:", predicted_words)

    if expected in predicted_words:
        correct += 1

accuracy = (correct / total) * 100

print("\nAccuracy =", round(accuracy, 2), "%")

```

Output:

```

=====
N-GRAM LANGUAGE MODEL
=====

```

Top Unigrams:

```

<s> : 21
</s> : 21
is : 10
language : 10
learning : 8
artificial : 7
intelligence : 7
a : 6
natural : 6
the : 5

```

Top Bigrams:

```

('artificial', 'intelligence') : 7
('natural', 'language') : 6
('<s>', 'artificial') : 5
('machine', 'learning') : 5
('language', 'processing') : 5
('intelligence', 'is') : 4
('learning', 'is') : 4
('<s>', 'machine') : 3
('is', 'a') : 3
('<s>', 'deep') : 3

```

Top Trigrams:

```

('<s>', 'artificial', 'intelligence') : 5
('natural', 'language', 'processing') : 5
('artificial', 'intelligence', 'is') : 4
('<s>', 'machine', 'learning') : 3
('<s>', 'deep', 'learning') : 3

```


('deep', 'learning', 'is') : 3
('<s>', 'natural', 'language') : 3
('learning', 'is', 'a') : 2
('of', 'artificial', 'intelligence') : 2
('artificial', 'intelligence', '</s>') : 2

=====

PREDICTION

=====

Input sentence: Artificial intelligence is

N = 1

is	Probability = 0.0508
language	Probability = 0.0508
learning	Probability = 0.0406
artificial	Probability = 0.0355
intelligence	Probability = 0.0355

N = 2

a	Probability = 0.3000
useful	Probability = 0.2000
used	Probability = 0.2000
transforming	Probability = 0.1000
changing	Probability = 0.1000

N = 3

transforming	Probability = 0.2500
changing	Probability = 0.2500
useful	Probability = 0.2500
becoming	Probability = 0.2500

=====

UNSEEN N-GRAM TEST

=====

Probability of unseen trigram ('quantum', 'banana', 'computer') = 0

=====

PREDICTION ACCURACY

=====

Input: Artificial intelligence is

Expected: transforming

Predicted: ['transforming', 'changing', 'useful', 'becoming']

Input: Machine learning is

Expected: a

Predicted: ['a', 'used']

Input: Deep learning is

Expected: a

Predicted: ['a', 'used']

Input: Natural language processing is

Expected: a

Predicted: ['a', 'useful']

Input: Language models predict

Expected: the

Predicted: ['the']

Accuracy = 100.0 %

2.

Code:

```
import re

from collections import Counter

# -----

# Q2: BACKOFF AND DELETED INTERPOLATION

# -----

corpus = """

Machine learning can analyze data.

Machine learning can solve problems.

Machine learning is a branch of artificial intelligence.

Artificial intelligence can solve complex problems.

Artificial intelligence can analyze large data.

Deep learning is a type of machine learning.

Deep learning can process images.

Deep learning can process text.

Natural language processing is an application of artificial intelligence.

Natural language processing can analyze text.

Language models can predict words.

Language models can generate text.

A language model learns patterns from data.

Smart systems can predict the next word.

Smart systems can understand language.

"""

# -----

# TOKENIZATION

# -----

def tokenize(text):

    text = text.lower()

    return re.findall(r"\b[a-z]+\b", text)

sentences = re.split(r"[!?.]+", corpus)
```

```

tokenized_sentences = []

for sentence in sentences:

    words = tokenize(sentence)

    if words:

        tokenized_sentences.append(["<s>" + words + "</s>"])

# -----

# N-GRAM COUNTS

# -----

unigrams = Counter()

bigrams = Counter()

trigrams = Counter()

for sentence in tokenized_sentences:

    for word in sentence:

        unigrams[word] += 1

    for i in range(len(sentence) - 1):

        bigrams[(sentence[i], sentence[i + 1])] += 1

    for i in range(len(sentence) - 2):

        trigrams[

            (sentence[i], sentence[i + 1], sentence[i + 2])

        ] += 1

total_words = sum(unigrams.values())

# -----

# PROBABILITIES

# -----

def unigram_prob(word):

    return unigrams[word] / total_words

def bigram_prob(w1, w2):

    denominator = unigrams[w1]

    if denominator == 0:

        return 0

    return bigrams[(w1, w2)] / denominator

```

```

def trigram_prob(w1, w2, w3):
    denominator = bigrams[(w1, w2)]
    if denominator == 0:
        return 0
    return trigrams[(w1, w2, w3)] / denominator
# -----
# UNSMOOTHED MODEL
# -----
def unsmoothed_prediction(sentence, top_k=5):
    words = tokenize(sentence)
    if len(words) < 2:
        return []
    w1 = words[-2]
    w2 = words[-1]
    predictions = []
    for (x1, x2, word), count in trigrams.items():
        if x1 == w1 and x2 == w2:
            probability = trigram_prob(x1, x2, word)
            if word not in ["<s>", "</s>"]:
                predictions.append((word, probability))
    predictions.sort(key=lambda x: x[1], reverse=True)
    return predictions[:top_k]
# -----
# BACKOFF MODEL
# -----
def backoff_probability(w1, w2, word):
    # Step 1: Trigram
    if trigrams[(w1, w2, word)] > 0:
        return trigram_prob(w1, w2, word)
    # Step 2: Bigram
    if bigrams[(w2, word)] > 0:
        return bigram_prob(w2, word)

```

```

# Step 3: Unigram

if unigrams[word] > 0:
    return unigram_prob(word)

return 0

def backoff_prediction(sentence, top_k=5):
    words = tokenize(sentence)

    if len(words) < 2:
        return []

    w1 = words[-2]
    w2 = words[-1]

    predictions = []

    vocabulary = [
        word for word in unigrams
        if word not in ["<s>", "</s>"]
    ]

    for word in vocabulary:
        probability = backoff_probability(w1, w2, word)

        if probability > 0:
            predictions.append((word, probability))

    predictions.sort(key=lambda x: x[1], reverse=True)

    return predictions[:top_k]

# -----

# DELETED INTERPOLATION

# -----

lambda1 = 0.2
lambda2 = 0.3
lambda3 = 0.5

def interpolation_probability(w1, w2, word):
    p1 = unigram_prob(word)
    p2 = bigram_prob(w2, word)
    p3 = trigram_prob(w1, w2, word)

```

```

probability = (
    lambda1 * p1
    + lambda2 * p2
    + lambda3 * p3
)

return probability

def interpolation_prediction(sentence, top_k=5):
    words = tokenize(sentence)
    if len(words) < 2:
        return []
    w1 = words[-2]
    w2 = words[-1]
    predictions = []
    vocabulary = [
        word for word in unigrams
        if word not in ["<s>", "</s>"]
    ]
    for word in vocabulary:
        probability = interpolation_probability(
            w1, w2, word
        )
        predictions.append((word, probability))
    predictions.sort(key=lambda x: x[1], reverse=True)
    return predictions[:top_k]

# -----
# DISPLAY PREDICTIONS
# -----

test_sentence = "Machine learning can"

print("=" * 65)

print("Q2: BACKOFF AND INTERPOLATION LANGUAGE MODEL")

print("=" * 65)

print("\nInput:", test_sentence)

```

```

print("\n1. UNSMOOTHED TRIGRAM MODEL")

results = unsmoothed_prediction(test_sentence)

for word, probability in results:

    print(f"{word:15s} {probability:.4f}")

print("\n2. BACKOFF MODEL")

results = backoff_prediction(test_sentence)

for word, probability in results:

    print(f"{word:15s} {probability:.4f}")

print("\n3. DELETED INTERPOLATION MODEL")

results = interpolation_prediction(test_sentence)

for word, probability in results:

    print(f"{word:15s} {probability:.4f}")

# -----

# ZERO PROBABILITY TEST

# -----

print("\n" + "=" * 65)

print("ZERO PROBABILITY TEST")

print("=" * 65)

w1 = "machine"

w2 = "learning"

word = "quantum"

print("\nTrigram probability:")

print(

    f"P({word} | {w1}, {w2}) =",

    trigram_prob(w1, w2, word)

)

print("\nBackoff probability:")

print(

    f"P({word} | {w1}, {w2}) =",

    backoff_probability(w1, w2, word)

)

print("\nInterpolation probability:")

```

```

print(
    f"P({word} | {w1}, {w2}) =",
    interpolation_probability(w1, w2, word)
)

```

Output:

```

=====
Q2: BACKOFF AND INTERPOLATION LANGUAGE MODEL
=====

Input: Machine learning can

1. UNSMOOTHED TRIGRAM MODEL
process      0.5000
analyze      0.2500
solve        0.2500

2. BACKOFF MODEL
process      0.5000
analyze      0.2500
solve        0.2500
predict      0.1818
generate     0.0909

3. DELETED INTERPOLATION MODEL
process      0.3078
analyze      0.2117
solve        0.1828
predict      0.0578
generate     0.0289

=====
ZERO PROBABILITY TEST
=====

Trigram probability:
P(quantum | machine, learning) = 0.0

Backoff probability:
P(quantum | machine, learning) = 0

Interpolation probability:
P(quantum | machine, learning) = 0.0

```


3.

Code:

```
import re
import math
from collections import Counter
# -----
# Q3: ENTROPY-BASED EVALUATION
# -----
training_corpus = """
The cat sits on the mat.
The cat eats food.
The dog sits on the floor.
The dog eats food.
The boy reads a book.
The girl reads a book.
The boy plays with the dog.
The girl plays with the cat.
The cat likes the boy.
The dog likes the girl.
Natural language processing analyzes text.
Natural language processing uses language models.
Language models predict the next word.
Machine learning analyzes data.
Machine learning predicts patterns.
Artificial intelligence analyzes information.
"""
testing_sentences = [
    "The cat sits on the mat.",
    "The dog eats food.",
    "The boy reads a book.",
    "The quantum processor redesigned the architecture."
]
# -----
# TOKENIZATION
# -----
def tokenize(text):
    return re.findall(r"\b[a-z]+\b", text.lower())
# -----
# BUILD LANGUAGE MODEL
# -----
train_sentences = re.split(r"[!?!?]+", training_corpus)
tokenized_train = []
for sentence in train_sentences:
    words = tokenize(sentence)
    if words:
        tokenized_train.append(["<s>"] + words + ["</s>"])
unigrams = Counter()
bigrams = Counter()
trigrams = Counter()
for sentence in tokenized_train:
    for word in sentence:
        unigrams[word] += 1
    for i in range(len(sentence) - 1):
```

```

        bigrams[(sentence[i], sentence[i + 1])] += 1
    for i in range(len(sentence) - 2):
        trigrams[
            (sentence[i], sentence[i + 1], sentence[i + 2])
        ] += 1
total_words = sum(unigrams.values())
vocabulary = set(unigrams.keys())
V = len(vocabulary)
# -----
# UNSMOOTHED PROBABILITIES
# -----
def unigram_probability(word):
    return unigrams[word] / total_words
def bigram_probability(w1, w2):
    denominator = unigrams[w1]
    if denominator == 0:
        return 0
    return bigrams[(w1, w2)] / denominator
def trigram_probability(w1, w2, w3):
    denominator = bigrams[(w1, w2)]
    if denominator == 0:
        return 0
    return trigrams[(w1, w2, w3)] / denominator
# -----
# ADD-ONE SMOOTHING
# -----
def smoothed_bigram_probability(w1, w2):
    numerator = bigrams[(w1, w2)] + 1
    denominator = unigrams[w1] + V
    return numerator / denominator
def smoothed_trigram_probability(w1, w2, w3):
    numerator = trigrams[(w1, w2, w3)] + 1
    denominator = bigrams[(w1, w2)] + V
    return numerator / denominator
# -----
# ENTROPY FUNCTION
# -----
def calculate_entropy(sentence, model="unigram"):
    words = tokenize(sentence)
    if not words:
        return 0
    words = ["<s>"] + words
    log_probability_sum = 0
    count = 0
    for i in range(1, len(words)):
        if model == "unigram":
            probability = unigram_probability(words[i])
        elif model == "bigram":
            probability = bigram_probability(
                words[i - 1],
                words[i]
            )
        elif model == "trigram":
            if i < 2:
                continue
            probability = trigram_probability(

```

```

        words[i - 2],
        words[i - 1],
        words[i]
    )
elif model == "smoothed":
    if i < 2:
        probability = unigram_probability(words[i])
    else:
        probability = smoothed_trigram_probability(
            words[i - 2],
            words[i - 1],
            words[i]
        )
    else:
        raise ValueError("Invalid model")
# Avoid log(0)
if probability <= 0:
    probability = 1e-10
log_probability_sum += math.log2(probability)
count += 1
if count == 0:
    return 0
entropy = -log_probability_sum / count
return entropy
# -----
# CLASSIFICATION
# -----
def classify_entropy(entropy):
    if entropy < 5:
        return "Low predictive uncertainty"
    return "High predictive uncertainty"
# -----
# DISPLAY RESULTS
# -----
print("=" * 70)
print("Q3: ENTROPY-BASED LANGUAGE MODEL EVALUATION")
print("=" * 70)
for sentence in testing_sentences:
    print("\nSentence:")
    print(sentence)
    print("\nUnigram Entropy:")
    h1 = calculate_entropy(sentence, "unigram")
    print(round(h1, 4))
    print("Bigram Entropy:")
    h2 = calculate_entropy(sentence, "bigram")
    print(round(h2, 4))
    print("Trigram Entropy:")
    h3 = calculate_entropy(sentence, "trigram")
    print(round(h3, 4))
    print("Smoothed Trigram Entropy:")
    hs = calculate_entropy(sentence, "smoothed")
    print(round(hs, 4))
    print("\nClassification:")
    print("Unigram:",
          classify_entropy(h1))
    print("Bigram:",

```

```

        classify_entropy(h2))

    print("Trigram:",
        classify_entropy(h3))

    print("Smoothed:",
        classify_entropy(hs))
# -----
# COMPARISON
# -----
print("\n" + "=" * 70)
print("ENTROPY COMPARISON")
print("=" * 70)
for sentence in testing_sentences:
    h1 = calculate_entropy(sentence, "unigram")
    h2 = calculate_entropy(sentence, "bigram")
    h3 = calculate_entropy(sentence, "trigram")
    hs = calculate_entropy(sentence, "smoothed")
    print("\n", sentence)
    print(f"Unigram : {h1:.4f}")
    print(f"Bigram : {h2:.4f}")
    print(f"Trigram : {h3:.4f}")
    print(f"Smoothed: {hs:.4f}")

```

Output:

Q3: ENTROPY-BASED LANGUAGE MODEL EVALUATION

Sentence:

The cat sits on the mat.

Unigram Entropy:

4.791

Bigram Entropy:

1.4755

Trigram Entropy:

0.9474

Smoothed Trigram Entropy:

3.8131

Classification:

Unigram: Low predictive uncertainty

Bigram: Low predictive uncertainty

Trigram: Low predictive uncertainty

Smoothed: Low predictive uncertainty

Sentence:

The dog eats food.

Unigram Entropy:

4.7983

Bigram Entropy:

1.1914

Trigram Entropy:

1.2457

Smoothed Trigram Entropy:

3.7232

Classification:

Unigram: Low predictive uncertainty

Bigram: Low predictive uncertainty

Trigram: Low predictive uncertainty

Smoothed: Low predictive uncertainty

Sentence:

The boy reads a book.

Unigram Entropy:

5.0857

Bigram Entropy:

0.9531

Trigram Entropy:

0.9767

Smoothed Trigram Entropy:

3.7945

Classification:

Unigram: High predictive uncertainty

Bigram: Low predictive uncertainty

Trigram: Low predictive uncertainty

Smoothed: Low predictive uncertainty

Sentence:

The quantum processor redesigned the architecture.

Unigram Entropy:

23.0571

Bigram Entropy:

27.7957

Trigram Entropy:

33.2193

Smoothed Trigram Entropy:

4.8542

Classification:

Unigram: High predictive uncertainty

Bigram: High predictive uncertainty

Trigram: High predictive uncertainty

Smoothed: Low predictive uncertainty

ENTROPY COMPARISON

The cat sits on the mat.

Unigram : 4.7910

Bigram : 1.4755

Trigram : 0.9474

Smoothed: 3.8131

The dog eats food.

Unigram : 4.7983

Bigram : 1.1914

Trigram : 1.2457

Smoothed: 3.7232

The boy reads a book.

Unigram : 5.0857

Bigram : 0.9531

Trigram : 0.9767

Smoothed: 3.7945

The quantum processor redesigned the architecture.

Unigram : 23.0571

Bigram : 27.7957

Trigram : 33.2193

Smoothed: 4.8542

4.

Code:

```
import re
from collections import Counter, defaultdict
# -----
# Q4: COMPARATIVE POS TAGGING SYSTEM
# -----
# Small tagged training corpus.
# Format: (word, POS-tag)
training_data = [
    [
        ("I", "PRP"),
        ("book", "VB"),
        ("a", "DT"),
        ("ticket", "NN")
    ],
    [
        ("I", "PRP"),
        ("read", "VB"),
        ("a", "DT"),
        ("book", "NN")
    ],
    [
        ("The", "DT"),
        ("boy", "NN"),
        ("plays", "VB"),
        ("football", "NN")
    ],
    [
        ("The", "DT"),
        ("girl", "NN"),
        ("reads", "VB"),
        ("books", "NN")
    ],
    [
        ("The", "DT"),
        ("cat", "NN"),
        ("sits", "VB"),
        ("on", "IN"),
        ("the", "DT"),
        ("mat", "NN")
    ],
]
```

```
[
  ("The", "DT"),
  ("dog", "NN"),
  ("runs", "VB"),
  ("quickly", "RB")
],
[
  ("The", "DT"),
  ("young", "JJ"),
  ("boy", "NN"),
  ("runs", "VB")
],
[
  ("The", "DT"),
  ("small", "JJ"),
  ("cat", "NN"),
  ("eats", "VB"),
  ("food", "NN")
],
[
  ("She", "PRP"),
  ("reads", "VB"),
  ("a", "DT"),
  ("book", "NN")
],
[
  ("He", "PRP"),
  ("books", "VB"),
  ("a", "DT"),
  ("ticket", "NN")
],
[
  ("They", "PRP"),
  ("play", "VB"),
  ("football", "NN")
],
[
  ("The", "DT"),
  ("students", "NN"),
  ("study", "VB"),
  ("hard", "RB")
],
[
  ("Artificial", "JJ"),
  ("intelligence", "NN"),
  ("is", "VB"),
  ("powerful", "JJ")
],
[
  ("Machine", "NN"),
  ("learning", "NN"),
  ("is", "VB"),
  ("useful", "JJ")
],
[
```



```

        ("Natural", "JJ"),
        ("language", "NN"),
        ("processing", "NN"),
        ("is", "VB"),
        ("important", "JJ")
    ]
]
# -----
# 1. BUILD STATISTICAL INFORMATION
# -----
word_tag_counts = defaultdict(Counter)
tag_counts = Counter()
transition_counts = defaultdict(Counter)
for sentence in training_data:
    previous_tag = "<START>"
    for word, tag in sentence:
        word = word.lower()
        word_tag_counts[word][tag] += 1
        tag_counts[tag] += 1
        transition_counts[previous_tag][tag] += 1
        previous_tag = tag
# -----
# TAG SET
# -----
TAGS = list(tag_counts.keys())
# -----
# 2. RULE-BASED TAGGER
# -----
def rule_based_tag(word, previous_word=""):
    original = word
    word = word.lower()
    # Lexical dictionary
    dictionary = {
        "i": "PRP",
        "you": "PRP",
        "he": "PRP",
        "she": "PRP",
        "they": "PRP",
        "we": "PRP",
        "the": "DT",
        "a": "DT",
        "an": "DT",
        "on": "IN",
        "in": "IN",
        "with": "IN",
        "for": "IN",
        "and": "CC",
        "boy": "NN",
        "girl": "NN",
        "cat": "NN",
        "dog": "NN",
        "book": "NN",
        "ticket": "NN",
        "football": "NN",
        "food": "NN",
        "mat": "NN",
    }

```

```

    "young": "JJ",
    "small": "JJ",
    "natural": "JJ",
    "artificial": "JJ",
    "powerful": "JJ",
    "useful": "JJ",
    "important": "JJ",
    "read": "VB",
    "reads": "VB",
    "play": "VB",
    "plays": "VB",
    "runs": "VB",
    "eats": "VB",
    "study": "VB",
    "studies": "VB",
    "is": "VB",
    "are": "VB",
    "am": "VB",
    "quickly": "RB",
    "hard": "RB"
}
# Context-sensitive rule for "book"
if word == "book":
    if previous_word.lower() in [
        "i", "you", "he", "she", "they", "we"
    ]:
        return "VB"
    return "NN"
if word in dictionary:
    return dictionary[word]
# Number rule
if word.isdigit():
    return "CD"
# Adverb pattern
if word.endswith("ly"):
    return "RB"
# Adjective patterns
if word.endswith(("ous", "ful", "able", "al", "ive")):
    return "JJ"
# Verb patterns
if word.endswith(("ing", "ed")):
    return "VB"
if word.endswith("s"):
    return "NN"
# Default
return "NN"
def rule_based_tagger(sentence):
    words = re.findall(r"\b[a-zA-Z]+\b", sentence)
    result = []
    previous_word = ""
    for word in words:
        tag = rule_based_tag(
            word,
            previous_word
        )

```

```

        result.append((word, tag))
        previous_word = word
    return result
# -----
# 3. STOCHASTIC POS TAGGER
# -----
def stochastic_tag(word, previous_tag=None):
    word_lower = word.lower()
    # If word was seen during training
    if word_lower in word_tag_counts:
        possible_tags = word_tag_counts[word_lower]
        best_tag = None
        best_score = -1
        for tag, count in possible_tags.items():
            emission_probability = (
                count /
                sum(possible_tags.values())
            )
            if previous_tag is not None:
                transition_total = sum(
                    transition_counts[
                        previous_tag
                    ].values()
                )
                if transition_total > 0:
                    transition_probability = (
                        transition_counts[
                            previous_tag
                        ][tag]
                        /
                        transition_total
                    )
                else:
                    transition_probability = 0.01
            else:
                transition_probability = 1
            score = (
                emission_probability
                *
                transition_probability
            )
            if score > best_score:
                best_score = score
                best_tag = tag
        return best_tag
    # Unknown words use the rule-based strategy
    return rule_based_tag(word)
def stochastic_tagger(sentence):
    words = re.findall(
        r"\b[a-zA-Z]+\b",
        sentence
    )
    result = []
    previous_tag = "<START>"
    for word in words:

```

```

        tag = stochastic_tag(
            word,
            previous_tag
        )
        result.append((word, tag))
        previous_tag = tag
    return result
# -----
# 4. TRANSFORMATION-BASED TAGGER
# -----
def initial_tag(word):
    word_lower = word.lower()
    # Basic initial tagging
    if word_lower in ["i", "you", "he", "she", "they", "we"]:
        return "PRP"
    if word_lower in ["the", "a", "an"]:
        return "DT"
    if word_lower in ["on", "in", "with", "for"]:
        return "IN"
    if word_lower.endswith("ly"):
        return "RB"
    if word_lower.endswith("ing"):
        return "VBG"
    if word_lower.endswith("ed"):
        return "VBD"
    if word_lower.endswith(("ous", "ful", "able", "ive")):
        return "JJ"
    # Known word information
    if word_lower in word_tag_counts:
        return word_tag_counts[
            word_lower
        ].most_common(1)[0][0]
    return "NN"
def transformation_based_tagger(sentence):
    words = re.findall(
        r"\b[a-zA-Z]+\b",
        sentence
    )
    tags = [
        initial_tag(word)
        for word in words
    ]
# -----
# Transformation Rules
# -----
    for i in range(len(words)):
        word = words[i].lower()
        previous_word = (
            words[i - 1].lower()
            if i > 0
            else ""
        )
        # Rule 1:
        # "book" after pronoun is a verb
        if (
            word == "book"

```

```

        and previous_word
        in ["I", "you", "he", "she", "they", "we"]
    ):
        tags[i] = "VB"
    # Rule 2:
    # word after determiner is generally noun
    elif (
        i > 0
        and tags[i - 1] == "DT"
        and word != "book"
    ):
        tags[i] = "NN"
    # Rule 3:
    # Words ending in ly are adverbs
    elif word.endswith("ly"):
        tags[i] = "RB"
    # Rule 4:
    # ing words are verbs/participles
    elif word.endswith("ing"):
        tags[i] = "VBG"
    return list(zip(words, tags))
# -----
# 5. DISPLAY TAGGING RESULTS
# -----
def display_result(title, result):
    print("\n" + title)
    for word, tag in result:
        print(f"{word:15s} -> {tag}")
# -----
# TEST SENTENCES
# -----
test_sentences = [
    "I book a ticket.",
    "I read a book.",
    "The young boy runs quickly.",
    "The small cat eats food.",
    "Artificial intelligence is powerful."
]
print("=" * 70)
print("Q4: COMPARATIVE POS TAGGING SYSTEM")
print("=" * 70)
for sentence in test_sentences:
    print("\n" + "=" * 70)
    print("INPUT:", sentence)
    # Rule-based
    result1 = rule_based_tagger(sentence)
    display_result(
        "RULE-BASED TAGGER",
        result1
    )
    # Stochastic
    result2 = stochastic_tagger(sentence)
    display_result(
        "STOCHASTIC TAGGER",
        result2
    )

```

```

# Transformation-based
result3 = transformation_based_tagger(sentence)
display_result(
    "TRANSFORMATION-BASED TAGGER",
    result3
)
# -----
# 6. SIMPLE ACCURACY EVALUATION
# -----
def calculate_accuracy():
    total = 0
    correct_rule = 0
    correct_stochastic = 0
    correct_transformation = 0
    for sentence in training_data:
        text = " ".join(
            word for word, tag in sentence
        )
        gold_tags = [
            tag for word, tag in sentence
        ]
        rule_result = rule_based_tagger(text)
        stochastic_result = stochastic_tagger(text)
        transformation_result = (
            transformation_based_tagger(text)
        )
        rule_tags = [
            tag for word, tag in rule_result
        ]
        stochastic_tags = [
            tag for word, tag in stochastic_result
        ]
        transformation_tags = [
            tag for word, tag
            in transformation_result
        ]
        for i in range(len(gold_tags)):
            total += 1
            if rule_tags[i] == gold_tags[i]:
                correct_rule += 1
            if stochastic_tags[i] == gold_tags[i]:
                correct_stochastic += 1
            if transformation_tags[i] == gold_tags[i]:
                correct_transformation += 1
    return (
        correct_rule / total * 100,
        correct_stochastic / total * 100,
        correct_transformation / total * 100
    )
rule_acc, stochastic_acc, transformation_acc = (
    calculate_accuracy()
)
print("\n" + "=" * 70)
print("TAGGING ACCURACY")
print("=" * 70)
print(

```

```
    f"Rule-Based Accuracy      : {rule_acc:.2f}%"
)
print(
    f"Stochastic Accuracy      : {stochastic_acc:.2f}%"
)
print(
    f"Transformation-Based Accuracy: "
    f"{transformation_acc:.2f}%"
)
```

Output:

Q4: COMPARATIVE POS TAGGING SYSTEM

INPUT: I book a ticket.

RULE-BASED TAGGER

I -> PRP

book -> VB

a -> DT

ticket -> NN

STOCHASTIC TAGGER

I -> PRP

book -> VB

a -> DT

ticket -> NN

TRANSFORMATION-BASED TAGGER

I -> PRP

book -> VB

a -> DT

ticket -> NN

INPUT: I read a book.

RULE-BASED TAGGER

I -> PRP

read -> VB

a -> DT

book -> NN

STOCHASTIC TAGGER

I -> PRP

read -> VB

a -> DT

book -> NN

TRANSFORMATION-BASED TAGGER

I -> PRP

read -> VB

a -> DT

book -> NN

INPUT: The young boy runs quickly.

RULE-BASED TAGGER

The -> DT

young -> JJ

boy -> NN

runs -> VB

quickly -> RB

STOCHASTIC TAGGER

The -> DT

young -> JJ

boy -> NN

runs -> VB

quickly -> RB

TRANSFORMATION-BASED TAGGER

The -> DT

young -> NN

boy -> NN

runs -> VB

quickly -> RB

INPUT: The small cat eats food.

RULE-BASED TAGGER

The -> DT

small -> JJ

cat -> NN

eats -> VB

food -> NN

STOCHASTIC TAGGER

The -> DT

small -> JJ

cat -> NN

eats -> VB

food -> NN

TRANSFORMATION-BASED TAGGER

The -> DT

small -> NN

cat -> NN

eats -> VB

food -> NN

INPUT: Artificial intelligence is powerful.

RULE-BASED TAGGER

Artificial -> JJ

intelligence -> NN

is -> VB

powerful -> JJ

STOCHASTIC TAGGER

Artificial -> JJ

intelligence -> NN

is -> VB

powerful -> JJ

TRANSFORMATION-BASED TAGGER

Artificial -> JJ

intelligence -> NN

is -> VB

powerful -> JJ

TAGGING ACCURACY

Rule-Based Accuracy : 93.65%

Stochastic Accuracy : 100.00%

Transformation-Based Accuracy: 92.06%